Contents

Guide 3 — The Search Engine (token parser + SQLite FTS5), the simple version

How Mobile Match Algeria turns a free-text search like *"djezzy 5gb unlimited prepaid streaming"* into the right plans, in plain words, with a worked example and ready answers for the jury.

---

1. The idea, in three sentences

The search bar lets the user type the way they think, in any of the three languages. The engine splits the query into two parts: the words it **recognises** (operator, data, price, network, plan type, unlimited) become **exact database filters**, and the **leftover words** go to a small **full-text index** that ranks plans by relevance. It then shows the user, as chips, exactly how the query was understood, so nothing is a black box.

---

2. How it works — five steps

1. **Read the query:** lowercase it and scan it with a few small patterns (regexes).
2. **Pull out the structured tokens:** operator (`djezzy`), data (`5gb`), price (`300da`), network (`5g`), plan type (`prepaid`), and the keyword `unlimited`.
3. **Turn the tokens into a database filter:** "djezzy" fixes the operator, "5 gb" means *about 5 GB* (or unlimited), "300 da" means *around 300 DA*, "5g" the network, "prepaid" the type, "unlimited" means unlimited calls or SMS.
4. **Send the leftover words to full-text search:** any word left over (here, "streaming") is matched against an indexed copy of every plan's name and features, **ranked by relevance** (BM25, the same scoring search engines use).
5. **Keep the plans that pass both** the structured filter **and** the text match, best match first, and show the parsed tokens as **"Read as" chips**.

---

3. The two engines behind one search bar

- **The token parser (exact, transparent).** A short list of patterns turns recognised words into precise filters. It is deterministic: the same query always parses the same way, and the user sees the result as chips.
- **Full-text search with BM25 (flexible, ranked).** The leftover words are handed to **SQLite's built-in FTS5** index. Each word is **prefix-matched** (so "stream" finds "streaming"), accents are folded (so

"donnees" finds "données"), and results come back ordered by **BM25 relevance**, a standard formula that favours rarer words and matches in the short name over the long features text.

The search bar is "fuzzy" on purpose: "5 gb" matches *roughly* 5 GB and "300 da" matches *around* 300 DA, because a user rarely knows the exact catalogue value. The **filter panel** (separate sliders) is the place for strict minimum-data / maximum-price control.

---

4. Where each part comes from

Nothing here is home-made scoring:

Part of the engine	What it does	Comes from
Token / faceted parse	recognised words become exact filters	the standard structured (faceted) product-search pattern (thesis Ch. 1 §1.4)
Full-text index	searches names and features for leftover words	**SQLite FTS5**, the database's built-in full-text engine
Relevance ranking	orders the matches best-first	**BM25** (Okapi BM25), the relevance function built into FTS5

What is *ours* is the configuration only: which words count as structured tokens, that "data" means *about* this much and "price" means *around* this much, and that the leftover text is ranked by FTS5. The scoring itself is BM25, computed by SQLite.

---

5. A worked example

Query typed by the user: `djezzy 5gb unlimited prepaid streaming`

Step 1 — the parser pulls out the tokens and leaves the rest as text:

Recognised	Token	Leftover text
`djezzy`	operator = Djezzy	
`5gb`	data ≈ 5 GB	
`unlimited`	unlimited calls or SMS	
`prepaid`	type = Prepaid	

Recognised	Token	Leftover text
`streaming`		→ “streaming”

Step 2 — the tokens become a database filter (Prisma `where`):

- operator = Djezzy
- data is unlimited **or** between 3.5 and 15 GB (the “about 5 GB” window)
- type = Prepaid
- calls **or** SMS unlimited

Step 3 — the leftover word goes to full-text search: `streaming*` is matched against the FTS5 index of plan names and features and ranked by BM25.

Step 4 — keep the overlap. The result is the set of **Djezzy prepaid plans of roughly 5 GB (or unlimited) with unlimited calls or SMS whose features mention streaming**, with the most relevant first.

Step 5 — the UI shows the chips: `[Djezzy] [≈ 5 GB] [Unlimited] [Prepaid] [streaming]`, so the user can see and remove any part of how the query was read.

---

6. The files

- **search-tokens.ts** — the parser that produces the display chips.
- **api/offers/route.ts** — the API: the same patterns build the database filter and call the full-text search.
- **search-fts.ts** — builds the FTS5 index and runs the BM25 query.
- **offers/page.tsx** — the search bar and the “Read as” chips.

---

7. Jury questions — ready answers

- **Why not one big `LIKE` query?** A `LIKE` cannot rank results and is slow on long text. Splitting the query gives **exact, fast filters** for the recognised parts and a **relevance-ranked** full-text match for the words, so the result is both precise and flexible.
- **Is the ranking invented?** No. The ordering is **BM25**, the relevance function built into SQLite FTS5, the same family of scoring used by industrial search engines. We do not compute a score ourselves.
- **What if the query has no recognised words?** Then the whole query is treated as free text and handled entirely by the full-text search.
- **Why “about 5 GB” and not exactly 5 GB?** Users rarely know the exact catalogue value, so the search matches intent with a window; the filter panel offers strict minimum-data / maximum-price sliders when exact control is wanted.
- **Does it work in French and Arabic?** Yes. The patterns accept Go / giga and the Arabic word for giga, and prépayé / postpayé; the full-text index folds accents, so “donnees” finds “données”.
- **How does the user know how the query was understood?** Every recognised token is shown as a chip under the search bar, and the user can remove any of them.

Next guide: notifications, or comparison and savings. Tell me which you want.